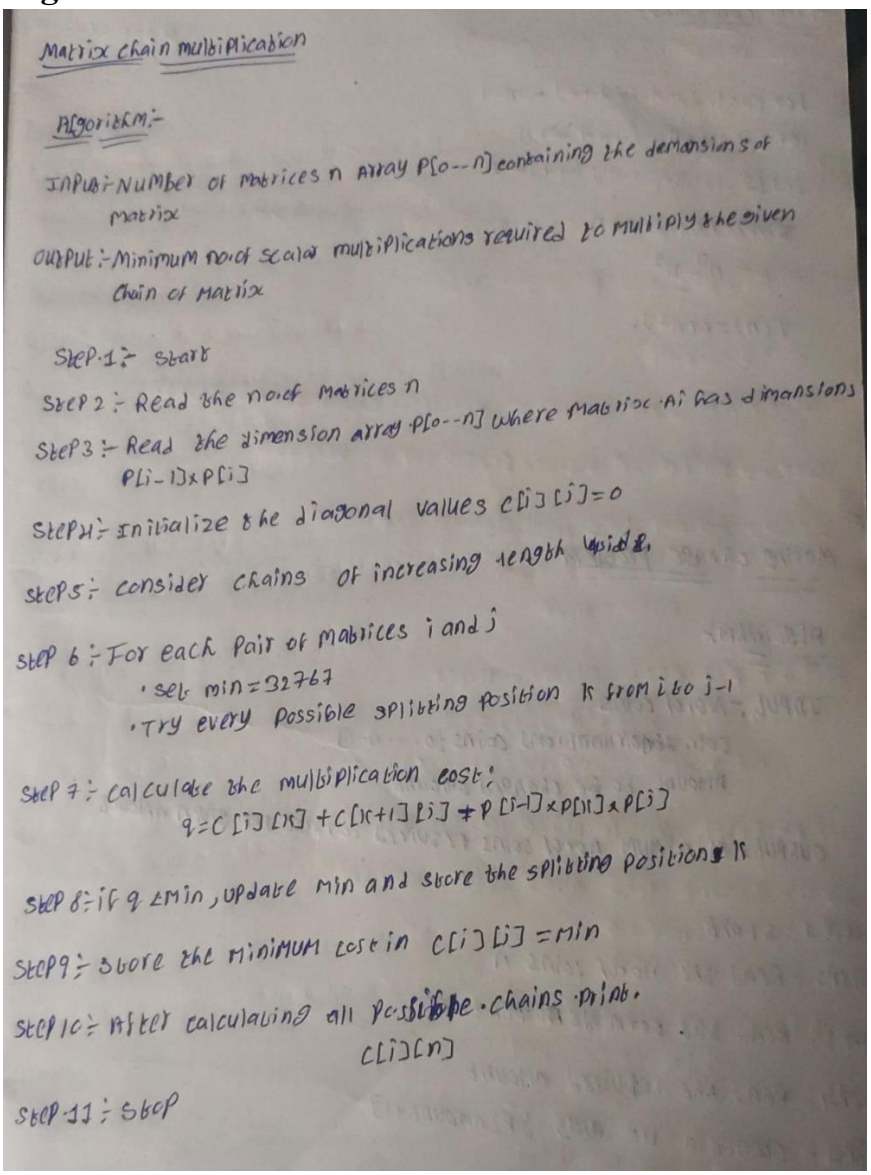
practical-6

Aim: Implementation of chain matrix multiplication using dynamic programming.

Matrix Chain multiplication

Algorithm:



Matrix chain multiplication

Algorithm:-

INPUT:- Number of matrices n Array P[0--n] containing the demensions of matrix

OUTPUT:- Minimum no.of scalar multiplications required to multiply the given chain of matrix

Step 1:- Start

Step 2:- Read the no.of matrices n

Step 3:- Read the dimension array P[0--n] where matrix Ai has dimensions P[i-1]xP[i]

Step 4:- Initialize the diagonal values C[i][i]=0

Step 5:- consider chains of increasing length l,

Step 6:- For each pair of matrices i and j
- set min=32767
- Try every possible splitting position k from i to j-1

Step 7:- calculate the multiplication cost;
q=C[i][k]+C[k+1][j]+P[i-1]xP[k]xP[j]

Step 8:- if q<min, update min and store the splitting position k

Step 9:- Store the minimum cost in C[i][j]=min

Step 10:- After calculating all possible chains print C[1][n]

Step 11:- Stop

Program:

```
#include <iostream> using
namespace std;

int main() {     int n, i, j, k, d, q, min;
int p[20], c[20][20], k_table[20][20];

   cout << "Enter number of matrices: ";
cin >> n;

   cout << "Enter the dimensions:" << endl;
for (i = 0; i <= n; i++)
   {        cin >>
p[i];
   }

   for (i = 1; i <= n; i++)
   {        c[i][i]
= 0;
   }

   for (d = 1; d <= n - 1; d++)
   {        for (i = 1; i < n - d;
i++)
      {           j = i +
d;           min =
32767;

          for (k = 1; k <= j - 1; k++)
          {              q = c[i][k] + c[k + 1][j] + p[i - 1] *
p[k] * p[j];
              if (q < min)
{                  min = q;
k_table[i][j] = k;
              }
          }
          c[i][j] =
min;
      }
```

```
    }

    cout << "\nMinimum number of multiplications = " << c[1][n] << endl;

    return 0;
}
```

Output:

```
Output

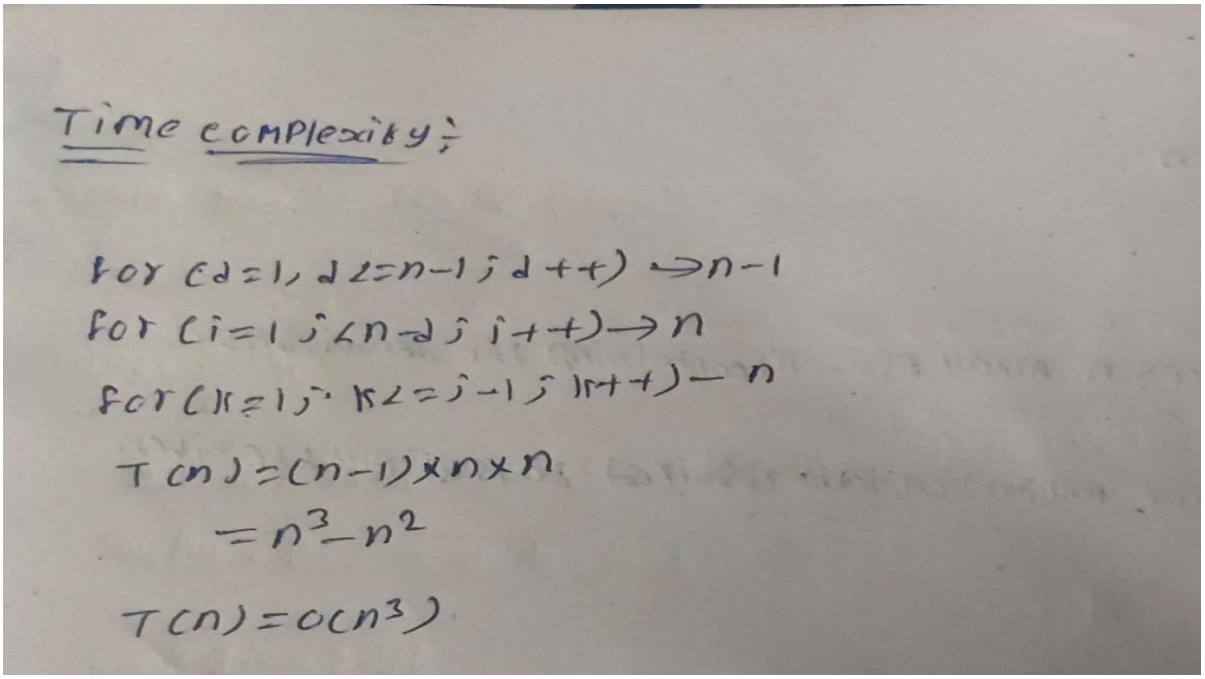
Enter number of matrices: 4
Enter the dimensions:
10 20
20 30
30 40

Minimum number of multiplications = 19000

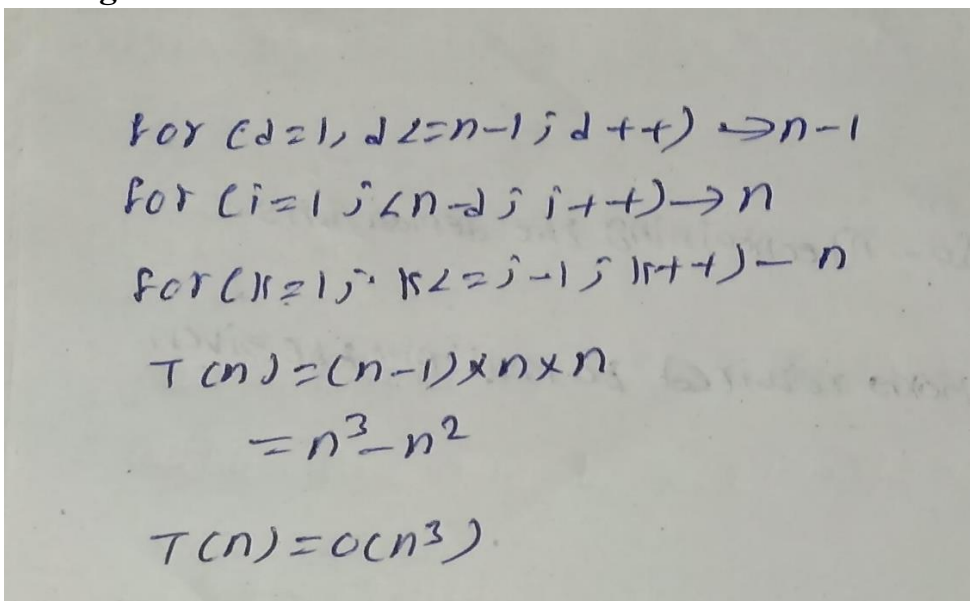

=== Code Execution Successful ===
```

Time Complexity:

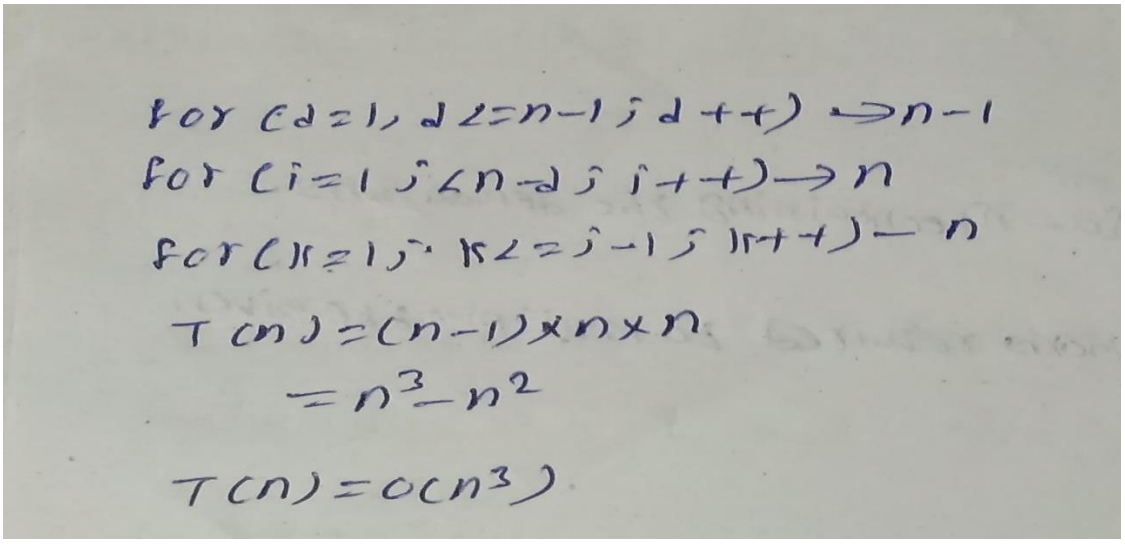
Best Case:



Average Case:



Worst Case:



Conclusion:

The implementation and time analysis of Matrix Chain Multiplication using Dynamic Programming helps us understand its efficiency. The algorithm builds a DP table and considers different ways of splitting the matrix chain to find the minimum number of scalar multiplications. It has a time complexity of $O(n^3)$ in the best, average, and worst cases. Therefore, Dynamic Programming provides an efficient and systematic solution for solving the Matrix Chain Multiplication problem.